

05506026 Digital Image Processing (Java)*Lab 7***Objectives**

1. Understand scale an image up and down using Nearest-Neighbour interpolation and Bilinear interpolation methods.

Scaling Algorithms

Image scaling is one of common processes in digital image processing which easily causes aliasing in the output. This is because the number of pixels of the scaled image is changed during the processes.

```
scale_x = new_width / width  
scale_y = new_height / height
```

There are a lot of scaling algorithms which produce different results with different computational costs. In this lab, we will try the two simplest scaling algorithms.

Before implementing any algorithms, we should start with altering a few parts of `restoreToOriginal()` method. Since the previous implementation, we copy the entire colour data without changing the width and height. But if we implement scaling algorithms, we need to change the width and height back to the original as well as the colour data.

ImageManager.java

```

class ImageManager
{
    ...
    public void restoreToOriginal()
    {
        width = original.getWidth();
        height = original.getHeight();

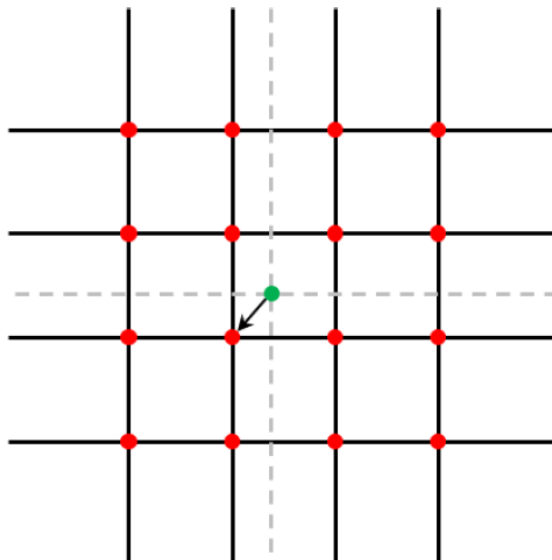
        img = new BufferedImage(width, height, img.getType());

        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                img.setRGB(x, y, original.getRGB(x, y));
            }
        }
    }
    ...
}

```

Nearest-Neighbour interpolation

Nearest-Neighbour interpolation takes the colour of the nearest integer pixel for the output pixel.



The code to perform nearest-neighbour interpolation is as below.

ImageManager.java

```

class ImageManager
{
    ...
    public void resizeNearestNeighbour(double scaleX, double scaleY)
    {
        if (img == null) return;

        int newWidth = (int) Math.round(width * scaleX);
        int newHeight = (int) Math.round(height * scaleY);

        BufferedImage tempBuf = new BufferedImage(newWidth, newHeight, img.getType());

        for (int y = 0; y < newHeight; y++)
        {
            for (int x = 0; x < newWidth; x++)
            {
                int xNearest = (int) Math.round(x / scaleX);
                int yNearest = (int) Math.round(y / scaleY);

                xNearest = xNearest >= width? width - 1: xNearest;
                xNearest = xNearest < 0? 0: xNearest;

                yNearest = yNearest >= height? height - 1: yNearest;
                yNearest = yNearest < 0? 0: yNearest;

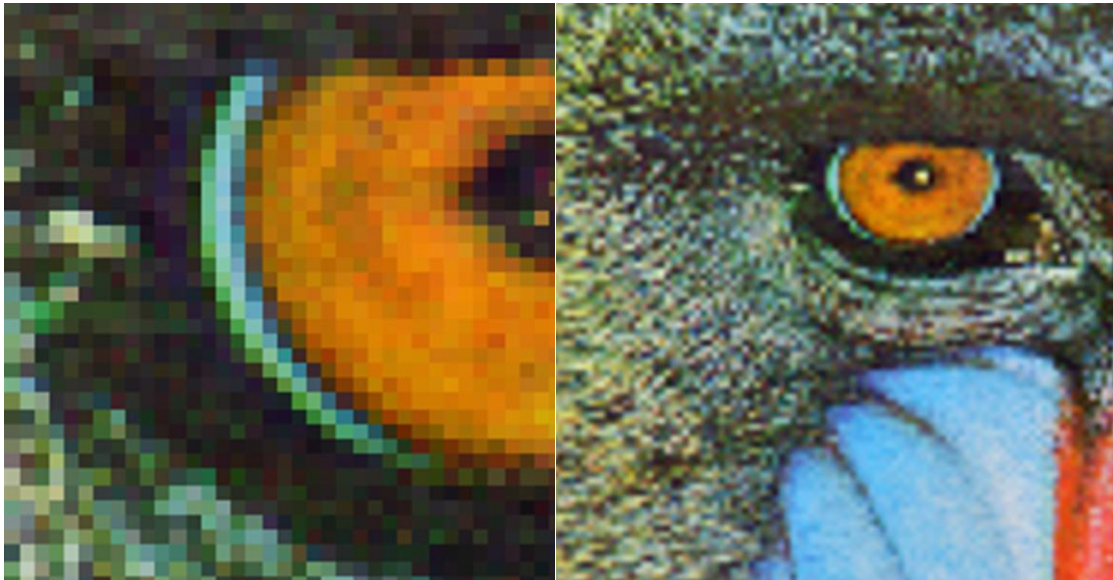
                tempBuf.setRGB(x, y, img.getRGB(xNearest, yNearest));
            }
        }

        img = new BufferedImage(newWidth, newHeight, img.getType());
        width = newWidth;
        height = newHeight;

        for (int y = 0; y < newHeight; y++)
        {
            for (int x = 0; x < newWidth; x++)
            {
                img.setRGB(x, y, tempBuf.getRGB(x, y));
            }
        }
    }
    ...
}

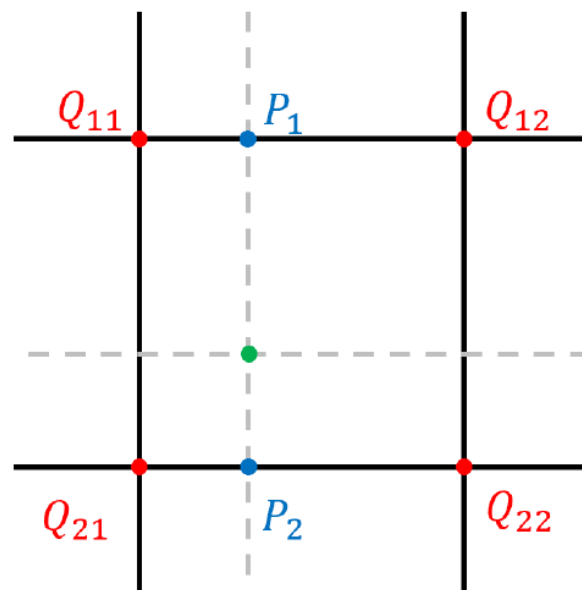
```

Below is Mandril.bmp with scale up and scale down. We can observe the pixels of these results.



Bilinear interpolation

This method interpolates the colour values using linear transition. After image is scaled, the value of the output pixel will be the weighted average of the neighbour pixels by their distance.



The code to perform bilinear interpolation is as below.

ImageManager.java

```

class ImageManager
{
    ...
    public void resizeBilinear(double scaleX, double scaleY)
    {
        if (img == null) return;

        int newWidth = (int) Math.round(width * scaleX);
        int newHeight = (int) Math.round(height * scaleY);

        BufferedImage tempBuf = new BufferedImage(newWidth, newHeight, img.getType());

        for (int y = 0; y < newHeight; y++)
        {
            for (int x = 0; x < newWidth; x++)
            {
                double oldX = x / scaleX;
                double oldY = y / scaleY;

                //get 4 coordinates
                int x1 = Math.min((int) Math.floor(oldX), width - 1);
                int y1 = Math.min((int) Math.floor(oldY), height - 1);
                int x2 = Math.min((int) Math.ceil(oldX), width - 1);
                int y2 = Math.min((int) Math.ceil(oldY), height - 1);

                //get colours
                int color11 = img.getRGB(x1, y1);
                int r11 = (color11 >> 16) & 0xff;
                int g11 = (color11 >> 8) & 0xff;
                int b11 = color11 & 0xff;

                int color12 = img.getRGB(x1, y2);
                int r12 = (color12 >> 16) & 0xff;
                int g12 = (color12 >> 8) & 0xff;
                int b12 = color12 & 0xff;

                int color21 = img.getRGB(x2, y1);
                int r21 = (color21 >> 16) & 0xff;
                int g21 = (color21 >> 8) & 0xff;
                int b21 = color21 & 0xff;

                int color22 = img.getRGB(x2, y2);
                int r22 = (color22 >> 16) & 0xff;

```

```

int b22 = color22 & 0xff;

//interpolate x
double P1r = (x2 - oldX) * r11 + (oldX - x1) * r21;
double P1g = (x2 - oldX) * g11 + (oldX - x1) * g21;
double P1b = (x2 - oldX) * b11 + (oldX - x1) * b21;

double P2r = (x2 - oldX) * r12 + (oldX - x1) * r22;
double P2g = (x2 - oldX) * g12 + (oldX - x1) * g22;
double P2b = (x2 - oldX) * b12 + (oldX - x1) * b22;

if (x1 == x2)
{
    P1r = r11;
    P1g = g11;
    P1b = b11;

    P2r = r22;
    P2g = g22;
    P2b = b22;
}

//interpolate y
double Pr = (y2 - oldY) * P1r + (oldY - y1) * P2r;
double Pg = (y2 - oldY) * P1g + (oldY - y1) * P2g;
double Pb = (y2 - oldY) * P1b + (oldY - y1) * P2b;

if (y1 == y2)
{
    Pr = P1r;
    Pg = P1g;
    Pb = P1b;
}

int r = (int) Math.round(Pr);
int g = (int) Math.round(Pg);
int b = (int) Math.round(Pb);

r = r > 255? 255: r;
r = r < 0? 0: r;

g = g > 255? 255: g;
g = g < 0? 0: g;

```

```

        b = b > 255? 255: b;
        b = b < 0? 0: b;

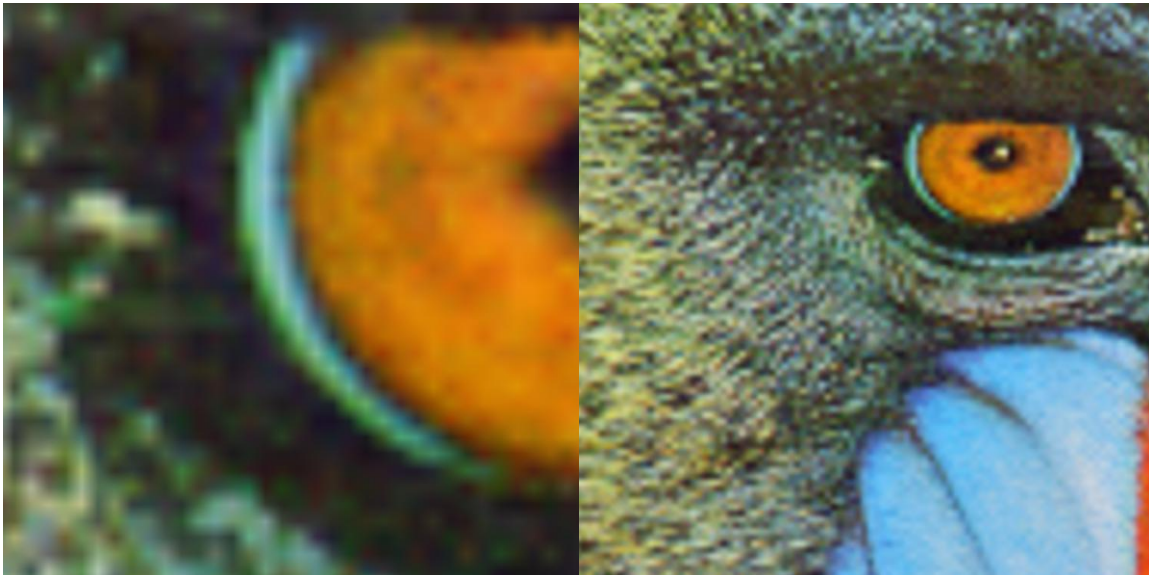
        int newColor = (r << 16) | (g << 8) | b;
        tempBuf.setRGB(x, y, newColor);
    }
}

img = new BufferedImage(newWidth, newHeight, img.getType());
width = newWidth;
height = newHeight;

for (int y = 0; y < newHeight; y++)
{
    for (int x = 0; x < newWidth; x++)
    {
        img.setRGB(x, y, tempBuf.getRGB(x, y));
    }
}
...
}

```

Below is Mandril.bmp with scale up and scale down. We can observe the pixels of these results.



Quest 011

Implement the method of bilinear interpolation and use it to perform scale 3.5 and 0.35.

The player who finishes this task will be awarded **10 EXP** and **1 move**.

Deadline: 7 September 2025